

第3章 表、栈和队列

建议学时:4-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 理解**抽象数据类型(ADT)**思想:先定义"做什么",再谈"怎么实现"
- 掌握表的两类实现:数组实现(倍增扩容)与链式实现(结点+指针),会对比两者复杂度
- 完成从 C 的 `malloc` 数组到 C++ `vector` 的迁移,理解容量与长度的区别
- 理解**迭代器失效**这一 C++ 独有的坑,并知道如何规避
- 能独立实现栈的数组版与链式版,并用栈解决括号平衡、后缀求值、中缀转后缀三个经典问题
- 能独立实现循环数组队列,说清空/满判断的两种方案
- 学完能独立实现中缀表达式计算器(栈应用综合),并会正确选用 `std::stack` / `std::queue`

💡 从 C 到 C++:本章主题如何迁移

本章是全书第一个"真正动手实现数据结构"的章节。算法直觉先给结论:表、栈、队列是后续所有复杂结构的积木——树的层序遍历要用队列,图的 DFS 要用栈,第6章的堆本质是数组,第5章的散列表是"数组+链表"的合体。本章学不透,后面每一章都会还债。

在 C 里,你已经会两种原始工具: `malloc` 出来的动态数组和 `struct` + 指针串起来的链表。C 的痛点是双重的:动态数组要自己管理"容量、长度、扩容"三件事,一次 `realloc` 写错就是内存泄漏或堆损坏;链表没有语言支持,每个操作都要手写指针舞蹈,一不小心就访问悬垂指针。更麻烦的是,C 里"数据结构"和"操作"是分离的——`sort` 一个数组和 `sort` 一个链表是两套完全不同的代码。

C++ 的解法分两层:**底层**用 `vector` 替你管理动态数组(自动扩容、自动释放,RAII 保证不泄漏),用 `list` 提供双向链表;**上层**用类把数据与操作封装成一个整体——这正是 ADT 思想,也是 C 的 `struct` 加函数指针表想模拟却始终别扭的东西。本章会手写一遍底层实现(理解原理),再告诉你标准库怎么用(工程实践)。

栈与队列部分,本章用三个经典应用(括号平衡、后缀求值、中缀转后缀)演示"数据结构服务于算法"的思维方式——它们是编译原理与表达式求值的原型,也是各大面试的高频题。

📦 前置知识自查

数学前置:本章几乎不需要新数学;倍增扩容的摊还分析直觉即可(严格分析在第11章)。

C 语言前置: `malloc` / `free` 动态数组、`struct` 与 `->` 指针访问、函数指针(可选)、`char*` 字符串遍历。

依赖的前面章节:第1章递归(栈的递归实现对照)、第2章大 O 记号(本章复杂度速查表直接用)。

🚀 本章学习路线

1. **第一阶段(1 小时)**:读 §3.1–§3.2,建立 ADT 概念,完成 vector 迁移练习
2. **第二阶段(1.5 小时)**:读 §3.3–§3.4,手写动态数组与单链表,上机验证
3. **第三阶段(1.5 小时)**:读 §3.5–§3.6,手写栈与队列,完成三个栈应用例题
4. **第四阶段(实验,1 小时)**:完成章末实验中缀表达式计算器
5. **验收标准**:能默写单链表插入/删除、循环队列、中缀转后缀,并说清每步复杂度

本章知识导图

- §3.1 ADT:抽象数据类型(接口与实现分离、从 C 的 struct+函数指针到类)
- §3.2 表 ADT 与从 C 的迁移(malloc 数组 → vector、倍增扩容、容量 vs 长度)
- §3.3 表的数组实现(手写 MyVector、vector 用法、迭代器与失效)
- §3.4 表的链式实现(结点与指针、手写单链表、vector 与 list 的选型)
- §3.5 栈(数组栈、括号平衡、后缀求值、中缀转后缀、std::stack)
- §3.6 队列(循环数组实现、空/满判断的两种方案、std::queue)
- §3.7 选型与工程实践(何时数组何时链表、栈模拟递归、调试技巧)

§3.1 ADT:抽象数据类型

3.1.1 什么是 ADT

抽象数据类型(ADT) = 一组操作 + 这些操作的语义, **不涉及实现细节**。例如"栈"这个 ADT 只有三个操作:push(入栈)、pop(出栈)、top(取栈顶),语义是"后进先出(LIFO)"。至于底层用数组还是链表、满时怎么办,属于"实现",与 ADT 无关。好处: **使用者只依赖接口,实现可以随时替换**——把数组栈换成链式栈,使用方代码一行不用改。

3.1.2 从 C 的 struct+函数指针到 C++ 的类

C 语言没有类,模拟 ADT 只能用 struct 存数据 + 函数指针表存操作:

C 的写法(struct + 函数指针)	C++ 的写法(class 封装)
<pre>struct Stack { int *a; int top; int cap; };</pre>	<pre>class Stack { vector<int> a; ... };</pre>
<pre>stack_init(&s); stack_push(&s, x);</pre> 每个操作传 &s	<pre>s.push(x);</pre> 数据与操作天然绑定
使用者能"意外"访问 <code>s.a[0]</code> 破坏结构	<code>private</code> 成员从语言层面禁止越界访问
忘记 <code>stack_destroy</code> 即泄漏,没有提醒	析构函数自动释放,RAII 不依赖使用者自觉

结论:ADT 的思想两种语言都能表达,C++ 的类是它的天然载体——本书所有数据结构都以"类"的形式呈现。

§3.2 表 ADT 与从 C 的迁移

3.2.1 表的基本操作

表(list)ADT:线性有序元素的集合,常用操作——按下标访问、按值查找、任意位置插入/删除、遍历。**同一个 ADT,两种截然不同的实现**(数组与链表),各有胜负,本章的主线就是这张对照表:

操作	数组实现(vector)	链式实现(list)
按下标访问	$O(1)$	$O(n)$
按值查找	$O(n)$	$O(n)$
尾部插入	平均 $O(1)$	$O(1)$
头部插入	$O(n)$	$O(1)$
任意位置插入/删除	$O(n)$ (移动元素)	$O(1)$ (已定位后)

3.2.2 C 的 malloc 数组 → C++ 的 vector

回忆 C 里管理动态数组的三件套:记录长度、记录容量、`realloc` 扩容。C++ 的 `vector` 把这套动作封装成一个类型: `size()` 返回当前元素个数(长度), `capacity()` 返回已分配空间能容纳的个数(容量),两者**不是一回事**——这是 C 程序员迁移时最需要建立的新概念。

C 的写法	C++ 的写法
<code>int *a = malloc(n * sizeof(int));</code>	<code>vector<int> a(n);</code>
<code>a[len++] = x;</code> (自己管长度)	<code>a.push_back(x);</code>
<code>realloc</code> 手动扩容(易错)	自动扩容,倍增策略
<code>free(a);</code> (忘记即泄漏)	离开作用域自动释放
<code>a[i]</code> 越界 = 未定义行为	<code>a.at(i)</code> 越界抛异常

倍增扩容:当 `push_back` 时容量已满,vector 分配一块**2 倍**大小的新内存,拷贝全部旧元素,释放旧内存。直觉上"每次扩容都是 $O(n)$,那插入怎么还说平均 $O(1)$?"——因为扩容发生得越来越少:容量从 1 倍增到 n 需要 $\log_2 n$ 次扩容,总拷贝代价 $n + n/2 + n/4 + \dots \approx 2n$,摊到 n 次插入上平均每次 $O(1)$ 。严格的摊还分析见第11章,现在先记住结论: **倍增使得尾部插入平均 $O(1)$** 。

⚠ 误区 1:capacity 与 size 混淆

`reserve(100)` 只预分配内存(改 capacity),元素个数仍是 0;要制造 100 个元素必须用 `resize(100)` 或插入。新手常写 `a.reserve(n); a[i] = x;` 然后崩溃——此时 `a[i]` 访问的是**不存在的元素**(越界)。口诀:reserve 管"房间",resize 管"住客"。

§3.3 表的数组实现

3.3.1 手写动态数组:搞懂倍增

示例 3-1 手写 MyVector:倍增扩容

```
// 手写动态数组:容量不足时倍增(2 倍)拷贝
class MyVector {
    int* data_ = nullptr;    // 元素存储
    size_t size_ = 0;        // 元素个数(长度)
    size_t cap_ = 0;         // 已分配容量
public:
    ~MyVector() { delete[] data_; }    // RAII:自动释放
    void push_back(int x) {
        if (size_ == cap_) {           // 满了:倍增
            size_t newCap = cap_ == 0 ? 1 : cap_ * 2;
            int* p = new int[newCap];
            for (size_t i = 0; i < size_; ++i) p[i] = data_[i];
            delete[] data_;
            data_ = p;
            cap_ = newCap;
        }
        data_[size_++] = x;
    }
    int& operator[](size_t i) { return data_[i]; }    // 按下标访问
    size_t size() const { return size_; }
    size_t capacity() const { return cap_; }
};
```

3.3.2 vector 常用操作速览

示例 3-2 vector 核心操作

```
#include <vector>
#include <iostream>
using namespace std;
int main() {
    vector<int> v;           // 空表
    v.push_back(3);          // 尾部插入:3
    v.push_back(1);          // 3 1
    v.insert(v.begin(), 9);   // 头部插入:9 3 1 (O(n),搬移全部元素)
    v.erase(v.begin() + 1);   // 删除下标 1:9 1 (O(n))
    v[1] = 5;                // 随机访问 O(1):9 5
    for (int x : v) cout << x << ' ';    // 范围 for 遍历
    cout << "\nsize=" << v.size() << " capacity=" << v.capacity();
    return 0;
}
```

3.3.3 迭代器与失效(本章最大的 C++ 专属坑)

迭代器(iterator)是"指向容器中位置"的对象,可以看作 C 指针的泛化。`v.begin()` 指向首元素, `v.end()` 指向尾后位置。

示例 3-3 用迭代器遍历(与 C 指针对照)

```
// C 指针遍历:
int a[5] = {1, 2, 3, 4, 5};
for (int* p = a; p != a + 5; ++p)    // 指针自增走到下一个元素
    printf("%d ", *p);

// C++ 迭代器遍历(结构完全一致:开始/结束/自增/解引用)
vector<int> v = {1, 2, 3, 4, 5};
for (auto it = v.begin(); it != v.end(); ++it)
    cout << *it << ' ';           // 迭代器就是"泛化的指针"
```

⚠ 误区 2:迭代器失效

vector 扩容时会**整体搬家**:旧内存被释放,之前保存的所有迭代器/指针立即失效。下面代码是未定义行为: `auto it = v.begin(); v.push_back(x); *it = 0;` // 若发生扩容, `it` 已指向被释放的内存!。规避三招:① 提前 `reserve` 足够容量;② 每次修改容器后重新获取迭代器;③ 只在下标遍历时才修改容器。同样的规则适用于 C 的 `realloc`——指针失效不是新问题,只是 C++ 把它包装得更隐蔽。

§3.4 表的链式实现

3.4.1 结点与指针

链表的原子单位是**结点(Node)**:一个数据域 + 一个指向下一个结点的指针。C 的 `struct Node { int val; struct Node *next; };` 与 C++ 的写法几乎一致(去掉了 `struct` 关键字而已)。链表不需要连续内存,插入/删除只改指针,不用搬移元素——代价是按下标访问要从头走 $O(n)$ 步。

3.4.2 手写单链表

示例 3-4 单链表:表头插入、查找、删除

```

struct Node {
    int val;
    Node* next;
    Node(int v, Node* n = nullptr) : val(v), next(n) {}
};

void insertFront(Node*& head, int x) {    // 表头插入 O(1)
    head = new Node(x, head);
}

Node* find(Node* head, int x) {          // 查找 O(n)
    while (head != nullptr && head->val != x)
        head = head->next;
    return head;                          // 找不到返回 nullptr
}

bool erase(Node*& head, int x) {          // 删除值为 x 的第一个结点
    if (head == nullptr) return false;
    if (head->val == x) {                  // 删表头是特例
        Node* t = head;
        head = head->next;
        delete t;
        return true;
    }
    Node* prev = head;                    // 其余情况:找前驱
    while (prev->next != nullptr && prev->next->val != x)
        prev = prev->next;
    if (prev->next == nullptr) return false;
    Node* t = prev->next;
    prev->next = t->next;                  // 前驱跳过目标结点
    delete t;
    return true;
}

```

★ 重点:插入删除的"定位"成本

说"链表插入删除 $O(1)$ "是有前提的: 已经定位到操作位置。定位本身(查找值、走到第 k 个结点)是 $O(n)$ 。所以"链表的插入比数组快"只在特定场景成立: 频繁头部操作、或已持有位置指针的遍历过程中。速查表里的" $O(1)$ "指的都是"已定位后"。

3.4.3 vector 与 list 的选型

C++ 标准库的 `std::list` 是双向链表,接口与 `vector` 高度一致(都有 `push_back`/`insert`/`erase`/迭代器)。选型口诀:

- 需要随机访问 → `vector`(链表按下标是 $O(n)$)
- 只在尾部增删 → `vector`(连续内存缓存友好,常数小)
- 频繁在头部/中间插入删除且已定位 → `list`
- 拿不准 → `vector`(工程统计:90% 的场景 `vector` 更快,链表赢在"无搬移"的场景才值得用)

示例 3-5 用 `std::list` 体会接口一致性

```

#include <list>
#include <iostream>
using namespace std;
int main() {
    list<int> l = {1, 2, 3};
    l.push_front(0);           // 头部插入 O(1), vector 没有这个接口!
    l.push_back(4);            // 尾部插入 O(1)
    auto it = l.begin();
    ++it; ++it;                // 走到第 2 个元素(下标访问是 O(n))
    l.insert(it, 9);           // 已定位后插入 O(1): 0 1 9 2 3 4
    for (int x : l) cout << x << ' '; // 双向链表也支持范围 for
    return 0;
}

```

工程建议:释放链表的内存

手写链表删除结点后忘记 `delete` 是最常见的泄漏源;整表销毁要在循环里逐个 `delete`。工程上更推荐用 `std::list` (自动管理)或在 `Node` 中用 `unique_ptr` (第4章树的实现会演示)。学习阶段手写是为了理解指针结构,生产代码请优先标准库。

§3.5 栈

3.5.1 栈 ADT 与数组实现

栈(stack):后进先出(LIFO)。三个操作——`push`(压入)、`pop`(弹出)、`top`(查看栈顶)。数组实现最简单:用 `vector` 存元素,用一个"栈顶下标"标记位置,只在尾部增删,三个操作全部 $O(1)$ 。

示例 3-6 数组栈(用 `vector` 实现)

```

class Stack {
    vector<int> a;           // 元素数组,栈顶在尾部
public:
    void push(int x) { a.push_back(x); }
    void pop() { a.pop_back(); } // 出栈:删除尾部
    int top() const { return a.back(); }
    bool empty() const { return a.empty(); }
    size_t size() const { return a.size(); }
};

```

注意与 C 的函数调用栈对照:递归深度过大时崩溃,本质就是这同一个结构——调用栈被 `push` 到溢出(第1章讲过)。栈也可以用单链表实现(表头做栈顶,头插头删 $O(1)$),接口完全不变——这正是 §3.1 说的 ADT 威力。

3.5.2 应用一:括号平衡检查

示例 3-7 括号平衡:三种括号匹配检查

```
// 判断字符串中 ( ) [ ] { } 是否配对且顺序正确
bool isBalanced(const string& s) {
    stack<char> st; // 标准库栈,元素是 char
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{')
            st.push(c); // 左括号入栈
        else if (c == ')' || c == ']' || c == '}') {
            if (st.empty()) return false; // 右括号多余
            char top = st.top();
            st.pop();
            if ((c == ')' && top != '(') || // 类型不匹配
                (c == ']' && top != '[') ||
                (c == '}' && top != '{'))
                return false;
        }
    }
    return st.empty(); // 结束时栈空 = 左括号无剩余
}
```

3.5.3 应用二:后缀表达式求值

后缀表达式(逆波兰记法)把运算符放在操作数之后: `6 5 2 3 + 8 * + 3 + *`,不需要括号。求值规则:遇数字入栈;遇运算符弹出两个操作数,算完把结果压回。这个"惰性计算"模式是栈最漂亮的用法。

示例 3-8 后缀表达式求值

```
// 输入:token 序列(数字串与运算符);输出:表达式值
int evalPostfix(const vector<string>& toks) {
    stack<int> st;
    for (const string& t : toks) {
        if (isdigit(t[0]) || (t.size() > 1 && t[0] == '-'))
            st.push(stoi(t)); // 数字入栈
        else {
            int b = st.top(); st.pop(); // 先弹的是右操作数!
            int a = st.top(); st.pop();
            if (t == "+") st.push(a + b);
            else if (t == "-") st.push(a - b);
            else if (t == "*") st.push(a * b);
            else st.push(a / b); // 只支持整除
        }
    }
    return st.top(); // 最后栈里只剩结果
}
```

⚠ 误区 3:操作数弹出顺序

后缀 `8 2 /` 是 $8/2=4$,但弹栈顺序是先弹 2 再弹 8——先弹的是右操作数。减法和除法对顺序敏感,写反了就是 $2/8$ 。口诀:栈顶是"后出现"的操作数,后出现的是右操作数。

3.5.4 应用三:中缀转后缀

人读的是中缀(`a+b*c`),机器算的是后缀。转换算法(调度场算法):遇数字直接输出;遇运算符,先把栈中**优先级不低于它的**运算符弹出输出,再压入自己;左括号直接入栈,右括号则弹出到左括号为止。这样保证高优先级先输出。

示例 3-9 中缀转后缀(单数字简化版)

```
int prec(char op) { return (op == '+' || op == '-') ? 1 : 2; }

string infixToPostfix(const string& s) {    // s 形如 "6+5*(2+3)",无空格
    string out;
    stack<char> st;
    for (char c : s) {
        if (isdigit(c)) out += c;          // 数字直接输出
        else if (c == '(') st.push(c);
        else if (c == ')') {
            while (st.top() != '(') { out += st.top(); st.pop(); }
            st.pop();                      // 丢弃左括号
        } else {                          // 运算符
            while (!st.empty() && st.top() != '(' &&
                prec(st.top()) >= prec(c)) {    // 栈顶优先级不低于当前
                out += st.top(); st.pop();
            }
            st.push(c);
        }
    }
    while (!st.empty()) { out += st.top(); st.pop(); }
    return out;
}
```

例题 3-1 手工走一遍中缀转后缀

输入 6+5*(2+3) :6 输出;+ 入栈;5 输出;* 入栈(+ 优先级低,留在栈中);(入栈;2 输出;+ 入栈;3 输出;) 弹出 +,再弹;
结束弹栈得 *、+。输出: 6523+*+。用 §3.5.3 求值:2+3=5,5*5=25,6+25=31。✓

3.5.5 标准库:std::stack

`std::stack` 是适配器:默认用 `deque` (双端队列)做底层,接口就是 `push/pop/top/empty/size`。本章示例 3-7 已经用过。想换底层可写 `stack<int, vector<int>>`——又是 ADT 思想的直接体现。

§3.6 队列

3.6.1 队列 ADT 与循环数组实现

队列(queue):先进先出(FIFO)。enqueue(入队)在队尾,poll/dequeue(出队)在队头。用数组实现有个经典问题:出队后前面空出的位置怎么办?如果每次出队都把全部元素前移,那是 $O(n)$;不移动、让队头指针"追赶"队尾指针,数组就要**循环**使用——下标到底后绕回 0。

示例 3-10 循环数组队列(用 count 记录元素数)

```
// front 指向队首元素,back 指向队尾下一个空位,count 记录元素数
class MyQueue {
    vector<int> data_;
    int front_ = 0, back_ = 0, count_ = 0;
public:
    MyQueue(int cap) : data_(cap) {}
    void push(int x) {                // 入队:假设未满(满的处理见误区4)
        data_[back_] = x;
        back_ = (back_ + 1) % int(data_.size());
        ++count_;
    }
    void pop() {                      // 出队
        front_ = (front_ + 1) % int(data_.size());
        --count_;
    }
    int front() const { return data_[front_]; }
    bool empty() const { return count_ == 0; }
    int size() const { return count_; }
};
```

★ 重点:空/满判断的两种方案

若用 front 与 back 两个下标判断空满,会遇到 $\text{front} == \text{back}$ 时"既可能空也可能满"的歧义。两种解法:① **计数法**(示例 3-10,多存一个 count,最简单);② **浪费一格法**(规定 back 永远不追赶 front,队满时留一个空格,满条件是 $(\text{back} + 1) \% \text{cap} == \text{front}$)。工程与面试都常考,两种都要会。

3.6.2 标准库:std::queue 与 deque

示例 3-11 std::queue 与 deque 用法

```
#include <queue>
#include <deque>
using namespace std;
int main() {
    queue<int> q;                // 默认基于 deque 的 FIFO 队列
    q.push(1); q.push(2); q.push(3);
    while (!q.empty()) {
        cout << q.front() << ' '; // 1 2 3(先进先出)
        q.pop();
    }
    deque<int> d = {1, 2, 3};    // 双端队列:头尾都可 O(1) 增删
    d.push_front(0);            // 0 1 2 3
    d.push_back(4);             // 0 1 2 3 4
    d.pop_front();              // 1 2 3 4
    return 0;
}
```

C 的手写循环队列	C++ 的标准库
自己维护 front/back 下标与取模	queue.push/pop/front 一行一个操作
空/满判断要写计数或浪费一格	自动扩容,永不"满"
free 忘记调用即泄漏	离开作用域自动释放

⚠ 误区 4:循环队列的下标溢出

示例 3-10 简化了"队满"检查——满时再 push 会覆盖旧元素。生产实现必须显式处理:计数法判 `count_ == cap` 满则拒绝/扩容,浪费一格法判 `(back_+1) % cap == front_`。另一个易错点是取模:back 先加 1 再对 capacity 取模,而不是对"当前元素数"取模。

§3.7 选型与工程实践

3.7.1 数组 or 链表?—一张决策表

场景	选择	理由
按序访问所有元素(遍历)	vector	连续内存,缓存友好,常数小
大量随机下标访问	vector	$O(1)$ vs $O(n)$
只做尾部增删(栈)	vector	均摊 $O(1)$,实现最简单
两端增删(队列)	deque / 循环数组	两头都 $O(1)$
遍历过程中删除元素	list	vector 会失效迭代器
内存极度受限且频繁插入	list	无扩容翻倍的峰值内存

3.7.2 栈模拟递归与调试技巧

任何递归都可以改写为"显式栈 + 循环"的迭代版(第4章树遍历会用到):函数调用栈压的是"返回地址 + 局部变量",显式栈压的是"待处理的任务"。迭代版不受系统栈深度限制,深度优先搜索(第9章)的大图遍历通常必须用迭代版。调试栈程序的两个技巧:① push 前打印"压入 x",pop 后打印"弹出 x",肉眼跟踪栈的演化;② 断言每个 pop 前 !empty(),把"逻辑错误"提前变成"崩溃点",定位速度翻倍。

🔧 工程建议:优先标准库,手写只为理解

本章手写了动态数组、链表、栈、队列四样东西——目的是让你看清每一层的复杂度来源。生产代码请一律使用 `vector/list/stack/queue/deque`:它们经过数十年打磨,边界处理(异常安全、自赋值、移动语义)远比自己写的健壮。手写实现的用武之地是"标准库没有的结构"(后几章的树、堆、散列表、图),那才是本书后续的主战场。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
按下标访问	vector(动态数组)	$O(1)$	$O(1)$
尾部插入(push_back)	vector	$O(1)$	$O(n)$ (扩容)
头部插入	vector	$O(n)$	$O(n)$
任意位置插入/删除	vector	$O(n)$	$O(n)$
按值查找	vector	$O(n)$	$O(n)$
按下标访问	list(链表)	$O(n)$	$O(n)$
按值查找	list	$O(n)$	$O(n)$
头部/尾部插入	list	$O(1)$	$O(1)$
已定位处插入/删除	list	$O(1)$	$O(1)$
push / pop / top	栈(数组或链表实现)	$O(1)$	$O(1)$
入队 / 出队 / 取队头	循环数组队列	$O(1)$	$O(1)$
入队 / 出队	链式队列	$O(1)$	$O(1)$
两端增删	deque(双端队列)	$O(1)$	$O(1)$
括号平衡检查	栈扫描一遍	$O(n)$	$O(n)$
后缀表达式求值	栈扫描一遍	$O(n)$	$O(n)$
中缀转后缀	调度场算法	$O(n)$	$O(n)$
vector 倍增扩容	n 次 push_back 总代价	摊还 $O(n)$, 平均每次 $O(1)$ (第11章证明)	

最小必会清单

- 能独立说出 ADT 的定义,并举例说明"接口与实现分离"的好处
- 能独立说出 vector 的 size 与 capacity 的区别,以及 reserve/resize 的语义
- 能独立默写倍增扩容的 push_back,并解释为什么平均 $O(1)$
- 能独立默写单链表的表头插入、查找、删除(含删表头特例)
- 能独立解释迭代器失效的原因,并说出三种规避手段
- 能独立默写数组栈,并用它解决括号平衡问题
- 能独立写出后缀表达式求值,并说清操作数的弹出顺序
- 能独立默写中缀转后缀算法(含运算符优先级处理)
- 能独立实现循环数组队列,并说出空/满判断的两种方案
- 能独立说出 vector 与 list 的选型规则(至少 4 条场景)

自测题

Q 1. vector 的 push_back 为什么说"平均 $O(1)$ "而最坏 $O(n)$?

✅ 参考答案

平时插入只写一个元素 $O(1)$; 仅当 size 达到 capacity 时才触发倍增扩容, 需拷贝全部 n 个元素 $O(n)$ 。扩容从 1 倍增到 n 共 $\log_2 n$ 次, 总拷贝 $n+n/2+\dots\approx 2n$, 摊到 n 次插入上平均 $O(1)$ 。

Q 2. `v.reserve(100)` 后 `v[50] = 1` 会怎样?为什么?

✅ 参考答案

未定义行为(通常崩溃)。`reserve` 只预分配内存, `size()` 仍为 0, `v[50]` 访问的是不存在的元素。应先 `resize(100)` 或改用 `push_back`。

Q 3. 单链表中删除"值为 x 的结点", 为什么表头要单独处理?怎样统一?

✅ 参考答案

普通结点删除需要前驱结点改指针, 而表头没有前驱。解法: ① 特判(本章示例 3-4); ② 引入哑结点(dummy), 表头前面永远站一个空结点, 删除逻辑统一; ③ 指针的指针 `Node**` 遍历"指向结点的指针", 统一修改。哑结点法在工程中最常用。

Q 4. 后缀表达式 `8 2 / 3 5 * +` 的值是多少?写出求值过程。

✅ 参考答案

8、2 入栈;/: 弹 2、8, $8/2=4$ 入栈; 3、5 入栈;*: 弹 5、3, $3*5=15$ 入栈;+: 弹 15、4, $4+15=19$ 。答案 19。注意先弹的是右操作数。

Q 5. 循环队列用 `front` 与 `back` 两个下标时, 为什么 `front == back` 会"既可能空也可能满"?两种解法是什么?

✅ 参考答案

队列空时 `front` 与 `back` 重合; 而队满时 `back` 绕一圈也追上 `front`, 两个状态无法区分。解法: ① 计数法——额外维护元素个数 `count`, 空满由 `count` 判断; ② 浪费一格法——规定 `back` 永不与 `front` 重合, 满条件是 $(back+1) \% cap == front$, 牺牲一个格子换取 $O(1)$ 判断。

Q 6. 中缀 `a+b*c-d` 转后缀的过程是什么?为什么 `*` 在 `+` 之前输出?

✅ 参考答案

`a` 输出; `+` 入栈; `b` 输出; 遇 `*`: 栈顶 `+` 优先级(1) 低于 `*`(2), 不弹出, `*` 入栈; `c` 输出; 遇 `-`: 栈顶 `*` 优先级不低于 `-`, 弹出 `*` 输出; 栈顶 `+` 优先级不低于 `-`, 弹出 `+` 输出; `-` 入栈; `d` 输出; 结束弹栈输出 `-`。结果 `abc*+d-`。`*` 先输出是因为它绑定得更紧(优先级高), 后缀表达式中运算符顺序即计算顺序。

章末实验题:中缀表达式计算器

实验 3:中缀表达式计算器(栈应用综合)

实验目标:实现一个命令行计算器:读入含 $+$ $-$ $*$ $/$ 与括号的中缀表达式,支持多位数,输出后缀表达式与计算结果。

实验要求:

- 任务 1:实现 tokenize,把字符串拆成数字与运算符序列,支持多位数(知识点:\$3.5.4 中缀转后缀的扩展)
- 任务 2:实现中缀转后缀,处理括号与优先级(知识点:\$3.5.4 调度场算法)
- 任务 3:实现后缀求值,注意弹出顺序(知识点:\$3.5.3)
- 任务 4:主循环读入表达式,输出后缀式与结果,遇到 `quit` 退出(知识点:\$3.5 栈应用综合)
- 任务 5:处理非法输入:除数为 0、括号不匹配时给出错误信息而非崩溃(知识点:\$3.5.2 括号平衡)

实验环境:C++17,MSVC 或 g++。编译: `cl /EHsc /std:c++17 lab3.cpp` 或 `g++ -std=c++17 lab3.cpp -o lab3`;运行 `lab3`。

第一步 tokenize 用指针扫描:连续数字拼成一个字符串 token;空格跳过;其余字符单独成 token

第二步 中缀转后缀直接复用示例 3-9 的骨架,把"单字符数字"换成"字符串 token";数字 token 用首字符 `isdigit` 判定

第三步 求值时先检查操作数栈是否够两个元素(非法输入),再检查除数是否为 0

第四步 括号匹配错误在转换阶段发现:弹栈遇空、或结束后栈里剩左括号,都报"括号不匹配"

✓ 参考答案(完整可运行程序,约 110 行,分两段)

```

// lab3.cpp — 中缀表达式计算器(第一段:词法与转换)
#include <iostream>
#include <string>
#include <vector>
#include <stack>
#include <cctype>
#include <stdexcept>
using namespace std;

// 任务 1:拆分 token(支持多位数与空格)
vector<string> tokenize(const string& s) {
    vector<string> toks;
    for (size_t i = 0; i < s.size(); i) {
        if (isspace((unsigned char)s[i])) { ++i; continue; }
        if (isdigit((unsigned char)s[i])) {
            string num;
            while (i < s.size() && isdigit((unsigned char)s[i]))
                num += s[i++];
            toks.push_back(num);
        } else {
            toks.push_back(string(1, s[i++]));
        }
    }
    return toks;
}

int prec(const string& op) { return (op == "+" || op == "-") ? 1 : 2; }

// 任务 2:中缀转后缀(调度场算法)
vector<string> infixToPostfix(const vector<string>& toks) {
    vector<string> out;
    stack<string> st;
    for (const string& t : toks) {
        if (isdigit((unsigned char)t[0]))
            out.push_back(t); // 数字直接输出
        else if (t == "(") st.push(t);
        else if (t == ")") {
            while (!st.empty() && st.top() != "(") {
                out.push_back(st.top()); st.pop();
            }
            if (st.empty()) throw runtime_error("括号不匹配");
            st.pop(); // 丢弃左括号
        } else { // 运算符
            while (!st.empty() && st.top() != "(" &&
                prec(st.top()) >= prec(t)) {
                out.push_back(st.top()); st.pop();
            }
            st.push(t);
        }
    }
    while (!st.empty()) {
        if (st.top() == "(") throw runtime_error("括号不匹配");
        out.push_back(st.top()); st.pop();
    }
    return out;
}

```



```
}
```

```
// 任务 3:后缀求值(注意弹出顺序)
```

lab3.cpp(第二段:求值与主循环)

```
int evalPostfix(const vector<string>& toks) {
    stack<int> st;
    for (const string& t : toks) {
        if (isdigit((unsigned char)t[0]))
            st.push(stoi(t));
        else {
            if (st.size() < 2) throw runtime_error("操作数不足");
            int b = st.top(); st.pop(); // 先弹的是右操作数
            int a = st.top(); st.pop();
            if (t == "+") st.push(a + b);
            else if (t == "-") st.push(a - b);
            else if (t == "*") st.push(a * b);
            else {
                if (b == 0) throw runtime_error("除数为 0");
                st.push(a / b);
            }
        }
    }
    if (st.size() != 1) throw runtime_error("表达式非法");
    return st.top();
}

int main() {
    cout << "中缀表达式计算器(支持 + - * / 与括号,输入 quit 退出)\n";
    string line;
    while (true) {
        cout << "> ";
        getline(cin, line);
        if (line == "quit") break;
        try {
            auto toks = tokenize(line);
            auto post = infixToPostfix(toks);
            int result = evalPostfix(post);
            cout << "后缀式: ";
            for (const string& t : post) cout << t << ' ';
            cout << "\n结果: " << result << endl;
        } catch (const exception& e) {
            cout << "错误: " << e.what() << endl;
        }
    }
    return 0;
}
```

示例输入输出:

```
> 6+5*(2+3)
后缀式: 6 5 2 3 + * +
结果: 31
> 12/4+7*2
后缀式: 12 4 / 7 2 * +
结果: 17
> (1+2)*3
后缀式: 1 2 + 3 *
结果: 9
> 5/0
错误: 除数为 0
> (1+2
错误: 括号不匹配
> quit
```

设计说明:① tokenize/转换/求值三阶段分离,每阶段只做一件事,出错信息定位到阶段——这是编译原理"词法→语法→语义"流水线的雏形;② 转换与求值都基于同一个栈结构,体现了 ADT"实现替换、接口不变"的思想(两个函数各自声明自己的 stack,互不干扰);③ 错误用异常 `throw` 统一报告,主循环一个 `catch` 兜底,避免每个函数各自打印错误导致"错误处理与逻辑纠缠";④ 运算符优先级集中在 `prec` 一个函数里,新增运算符(如幂 `^`)只需改这一处。

下一章预告:第4章 树——把"链式结点"推广为"有分支的结构",学习二叉树遍历、表达式树、二叉查找树与 AVL 平衡。第4章的表达式树与本章的中缀转后缀直接衔接;树的层序遍历将使用本章的队列。